Algorithm Analysis And Data Structures

Prof. Dr. Ayla ŞAYLI

http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr

Chapter 7: Sorting

- Preliminaries
- Insertion Sort
- Shellsort
- Bubblesort
- Mergesort
- Quicksort
- Radix Sort

7.1. Preliminaries

- *Sorting* is a process that organizes a collection of data into either ascending or descending order.
- An *internal sort* requires that the collection of data fit entirely in the computer's main memory.
- An *external sort* can be used when the collection of data cannot fit in the computer's main memory all at once but must reside in secondary storage such as on a disk.
- We will analyze only internal sorting algorithms.

7.1. Preliminaries

- Any significant amount of computer output is generally arranged in some sorted order so that it can be interpreted.
- Sorting also has indirect uses. An initial sort of the data can significantly enhance the performance of an algorithm.
- Majority of programming projects use a sorting algorithm somewhere, and in many cases, the sorting cost determines the running time.
- A comparison-based sorting algorithm makes ordering decisions only on the basis of comparisons.

Insertion Sort

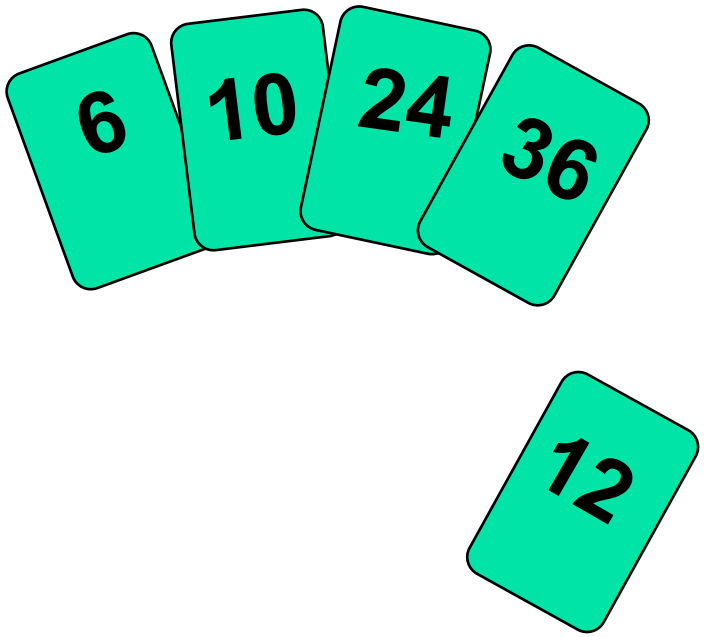
- Insertion sort is a simple sorting algorithm that is appropriate for small inputs.
  - Most common sorting technique used by card players.
- The list is divided into two parts: sorted and unsorted.
- In each pass, the first element of the unsorted part is picked up, transferred to the sorted sub-list, and inserted at the appropriate place.
- A list of *n* elements will take at most *n-1* passes to sort the data.

Insertion Sort

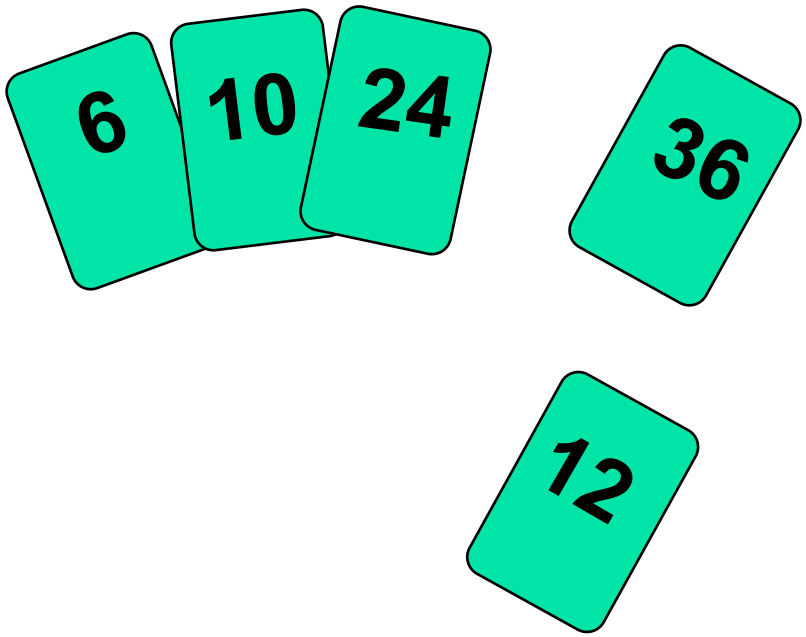
- Idea: like sorting a hand of playing cards
  - Start with an empty left hand and the cards facing down on the table.
  - Remove one card at a time from the table, and insert it into the correct position in the left hand
    - compare it with each of the cards already in the hand, from right to left
  - The cards held in the left hand are sorted
    - these cards were originally the top cards of the pile on the table

Insertion Sort

To insert 12, we need to make room for it by moving first 36 and then 24.

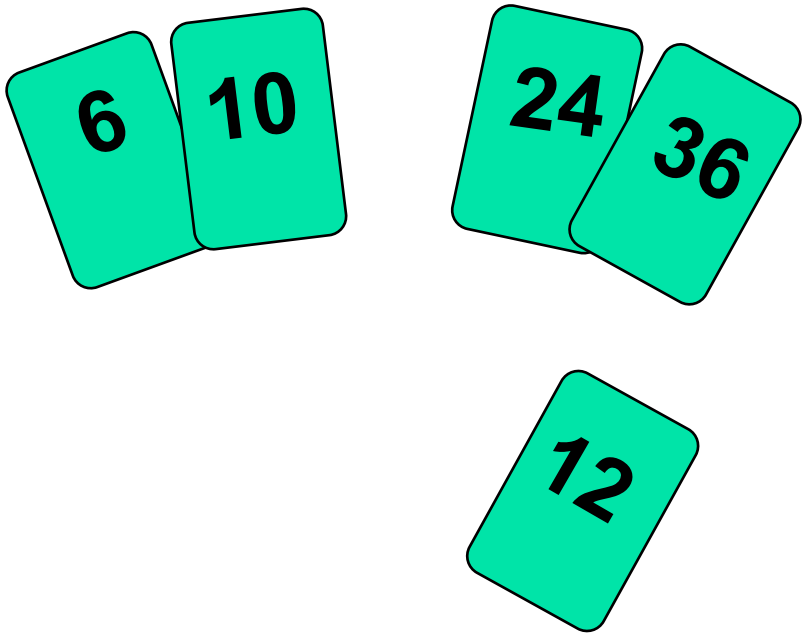


Insertion Sort



Designed by Ayla SAYLI and Armagan ELIBOL

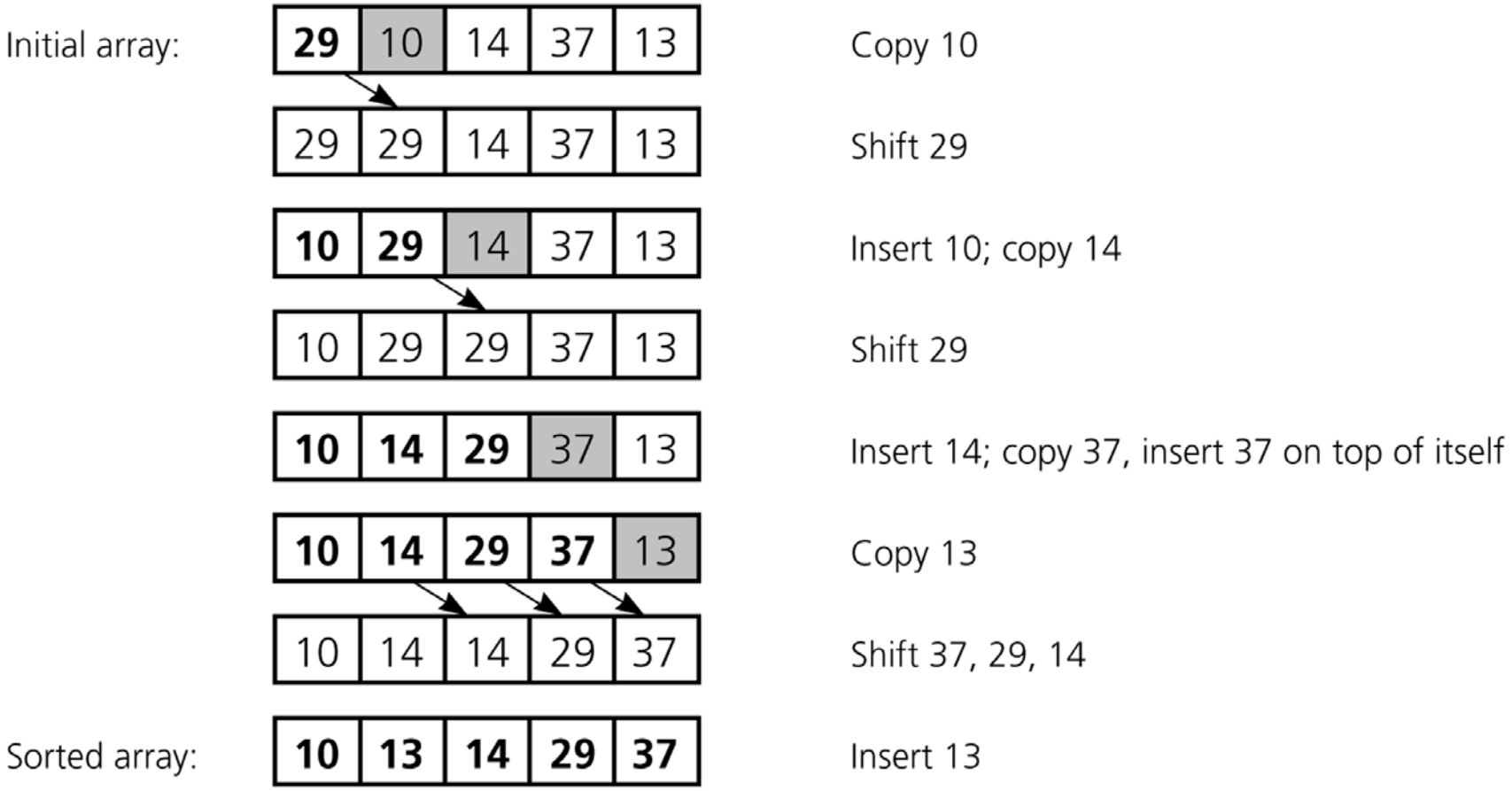
Insertion Sort



Designed by Ayla SAYLI and Armagan ELIBOL

Insertion Sort

Example 2



Insertion Sort

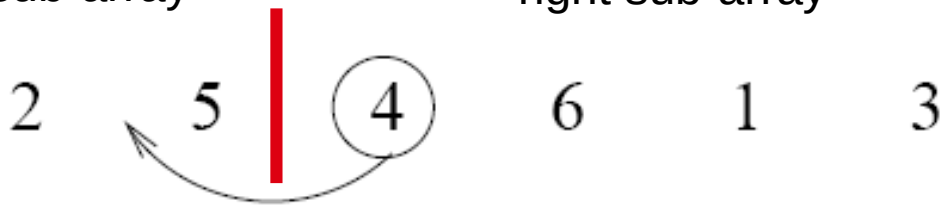
input array

5 2 4 6 1 3

at each iteration, the array is divided in two sub-arrays:

left sub-array

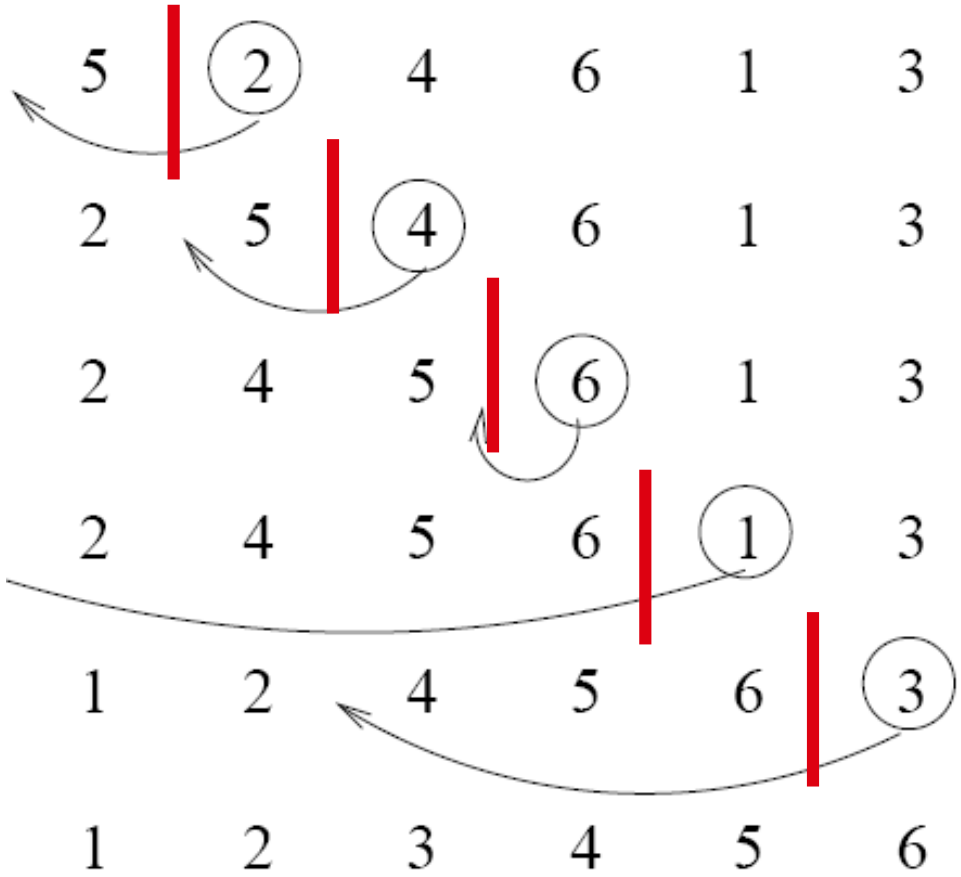
right sub-array

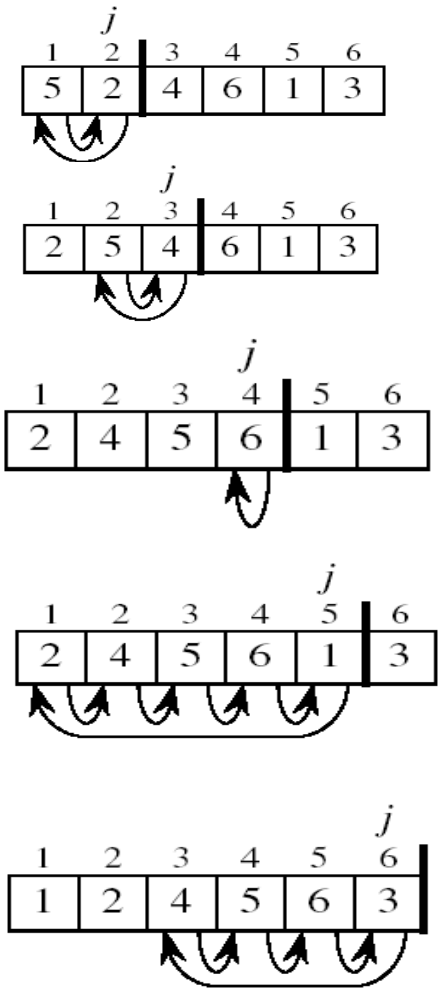


sorted

unsorted

Insertion Sort





Designed by Ayla SAYLI and Armagan ELIBOL

C Code of Insertion Sort

```
Void InsertionSort( int A[ ], int N )
{  int j, P, Tmp;

   for( P = 1; P < N; P++ )
   {    Tmp = A[ P ];
        for( j = P; j > 0 && A[ j - 1 ] > Tmp; j-- )
                A[ j ] = A[ j - 1 ];
        A[ j ] = Tmp;
   }
}
```

Figure 7.2 Insertion sort routine

Analysis of Insertion Sort

Which running time will be used to characterize this algorithm?

- Best, worst or average?

- Worst:
  - Longest running time (the upper limit for the algorithm)
  - It is guaranteed that the algorithm will not be worse than this.
- Sometimes we are interested in average case. But there are some problems with the average case.
  - It is difficult to figure out the average case. i.e. what is average input?
  - Are we going to assume all possible inputs are equally likely?
  - In fact for most algorithms average case is same as the worst case.

Analysis of Insertion Sort

- Running time depends on not only the size of the array but also the contents of the array.
- **Best-case: ➔ O(N)**
  - Array is already sorted in ascending order.
  - Inner loop will not be executed.
  - The number of moves: 2*(N-1) ➔ O(N)
  - The number of key comparisons: (N-1) ➔ O(N)
- **Worst-case: ➔ $O(N^2)$**
  - Array is in reverse order:
  - Inner loop is executed i-1 times, for i = 2,3, …, N
  - The number of moves: 2*(N-1)+(1+2+...+N-1)= 2*(N-1)+ N*(N-1)/2 ➔ $O(N^2)$
  - The number of key comparisons: (1+2+...+N-1)= N*(N-1)/2 ➔ $O(N^2)$
- **Average-case: ➔ $O(N^2)$**
  - We have to look at all possible initial data organizations.

So, Insertion Sort is $O(N^2)$

Shellsort

- It is named after its inventor, Donald Shell.
- Shellsort uses a sequence, h1, h2, h3,…, h_k, called the increment sequence.
- 2 ways of its application exist:

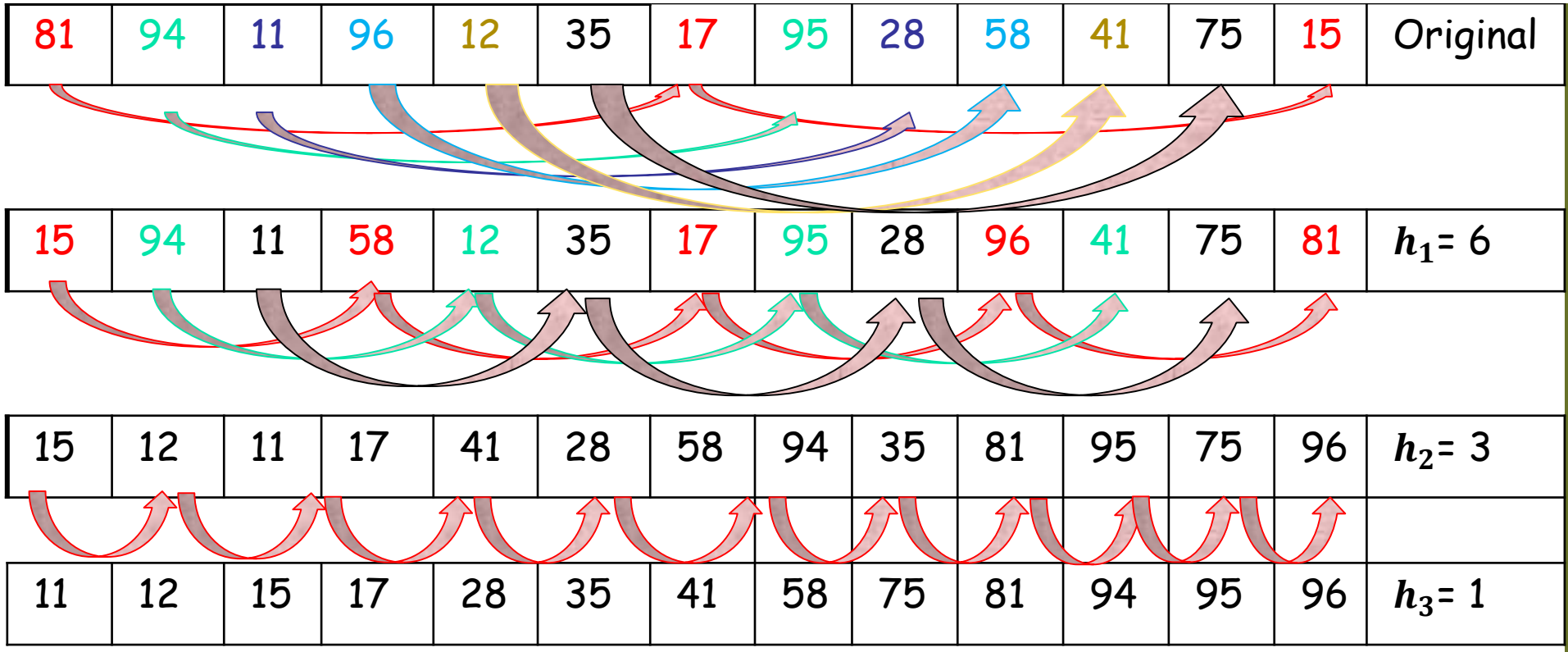
1 way: h_1 = ⌊Number of Elements / 2 ⌋
$h_2 = \lfloor h_1/2 \rfloor$, $h_3 = \lfloor h_2/2 \rfloor$ …
Any increment sequence will do as long as $h_k = 1$

2 way: h_1 = c (pozitive constant)
$h_2 = \lfloor h_1/2 \rfloor$, $h_3 = \lfloor h_2/2 \rfloor$ …
Any increment sequence will do as long as $h_k = 1$

Shellsort

Example 1: 81, 94, 11, 96, 12, 35, 17, 95, 28, 58, 41, 75, 15

1. way: $h_1 = \lfloor 13/2 \rfloor = 6$



C Code of Shellsort

```
void Shellsort(int A[ ], int N )
{       int i, j, Increment, Tmp;

        for( Inct = N / 2; Increment > 0; Increment /= 2 )
        for( i = Increment; i < N; i++ )
        {       Tmp = A[ i ];
                for( j = i; j >= Increment; j - = Increment )
                        if( Tmp < A[ j - Increment ] )
                                A[ j ] = A[ j - Increment ];
                        else
                                break;
                A[ j ] = Tmp;
        }
}
```

Figure 7.4 Shellsort routine

Analysis of Shellsort

- Running time depends on not only the size of the array but also the contents of the array.
- Shell's increments are not necessarily relatively prime and the smaller increment can have little effect.
  - Worst-case ➔ $\Theta(N^2)$ using Shell's increments
    - N/2 smallest elements requires at least $\sum_{i=1}^{N/2}(i-1)$ = $\Omega(N^2)$ work
    - A pass with increment h_k consists of h_k insertion sorts of about N/h_k elements. Therefore the total cost of a pass is $O(h_k(N/h_k)^2) = O(N^2/h_k)$.
    - Summing over all passes gives a tatl bound of $O(\sum_{i=1}^{t}(N^2/h_i) = O(N^2 \sum_{i=1}^{t}(1/h_1))$

      $= O(N^2)$
- Hibbard suggested a diffrent increment sequence which are in the form of 1, 3, 7, ..., 2^k-1.
  - Worst-case ➔ $\Theta(N^{\frac{3}{2}})$ using Hibbard's increments

(the proof is in Page 225)

Bubble Sort

- The list is divided into two sublists: sorted and unsorted.
- The smallest element is bubbled from the unsorted list and moved to the sorted sublist.
- After that, the wall moves one element ahead, increasing the number of sorted elements and decreasing the number of unsorted ones.
- Each time an element moves from the unsorted part to the sorted part one sort pass is completed.
- Given a list of n elements, bubble sort requires up to n-1 passes to sort the data.

Bubble Sort Algorithm

```
void BubbleSort(int *a, int n)
{ int sorted = 0;
  int i,j,last = n-1;

  for (i = 0; (i < last) && sorted==0; i++)
  {  sorted = 1;
     for (j=last; j > i; j--)
        if (a[j-1] > a[j])
        { swap(&a[j],&a[j-1]);
          sorted = 0; // signal exchange
        }
    }
}
```

Analysis of Bubble Sort

- ***Best-case:*** ➔ **O(n)**
  - Array is already sorted in ascending order.
  - The number of moves: 0 ➔ O(1)
  - The number of key comparisons: (n-1) ➔ O(n)
- ***Worst-case:*** ➔ **$O(n^2)$**
  - Array is in reverse order:
  - Outer loop is executed n-1 times,
  - The number of moves: $3*(1+2+...+n-1) = 3 * n*(n-1)/2$ ➔ $O(n^2)$
  - The number of key comparisons: $(1+2+...+n-1)= n*(n-1)/2$ ➔ $O(n^2)$
- **Average-case:** ➔ **$O(n^2)$**
  - We have to look at all possible initial data organizations.
- **So, Bubble Sort is $O(n^2)$**

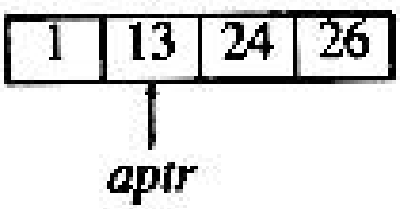
Heapsort

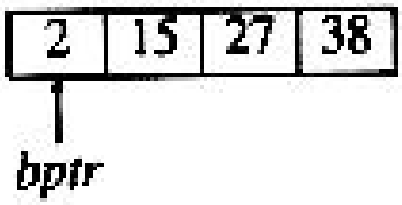
It can be given after data structure,
especially binary search trees...

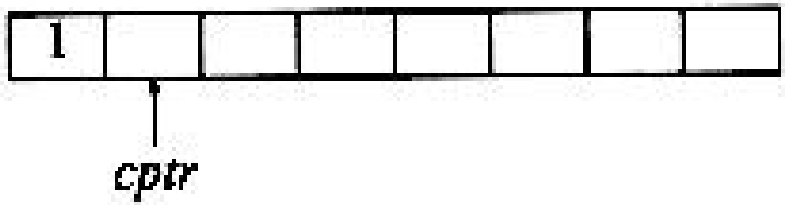
Mergesort

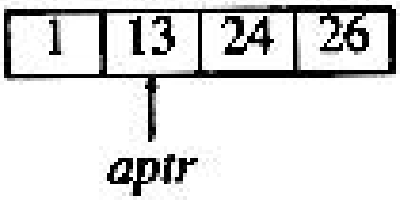
- The fundamental operation in this algorithm is merging two sorted lists.
- The basic merging algorithm takes two input arrays
- a and b, an output array c, and three counters, aptr, bptr, and cptr, which are initially set to the beginning of their respective arrays.
  - The smaller of a[aptr] and b[bptr] is copied to the next entry in c, and the appropriate counters are advanced. When either input list is exhausted, the remainder of the other list is copied to c.

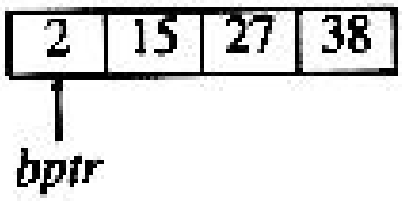
Mergesort

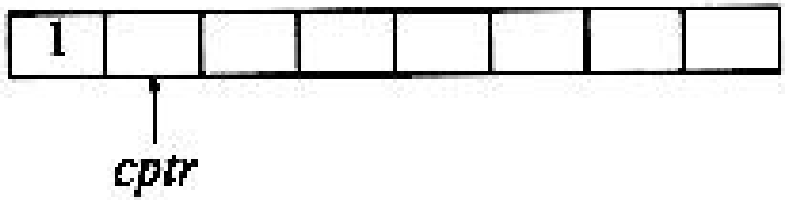

1
13
24
26
aptr

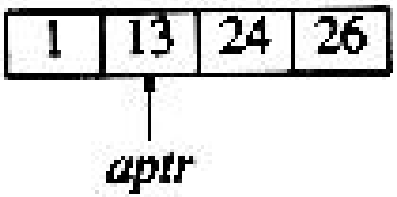

2
15
27
38
bptr

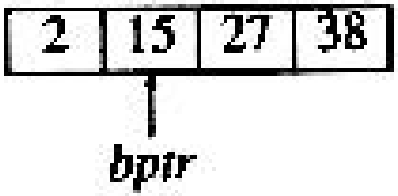

1
cptr

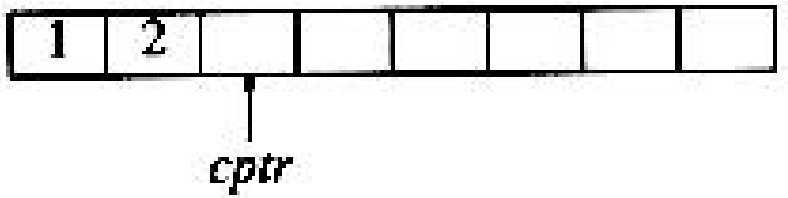

1
13
24
26
aptr


2
15
27
38
bptr

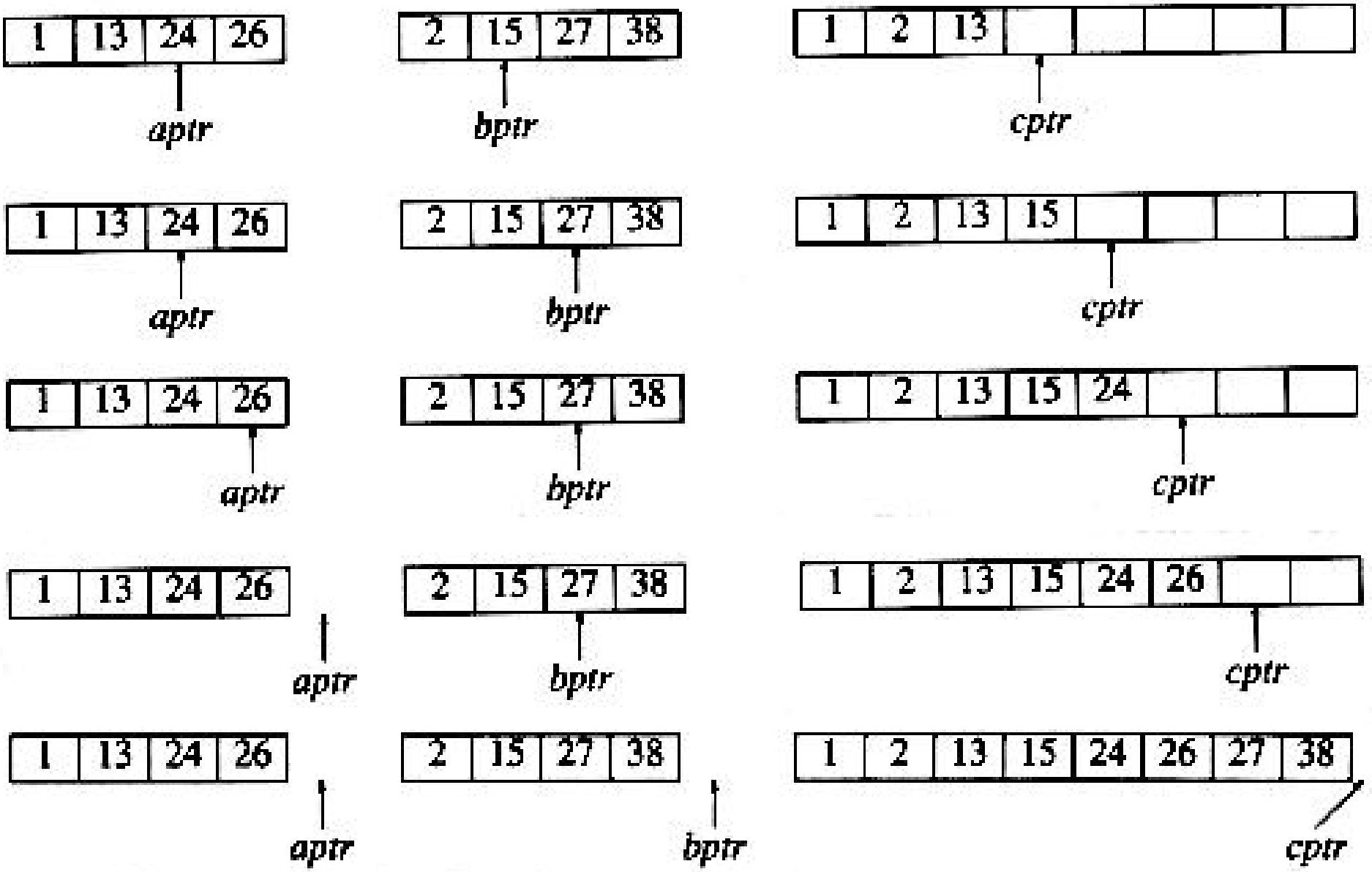

1
cptr


1
13
24
26
aptr


2
15
27
38
bptr


1
2
cptr

Mergesort


1 13 24 26
aptr
2 15 27 38
bptr
1 2 13
cptr
1 13 24 26
aptr
2 15 27 38
bptr
1 2 13 15
cptr
1 13 24 26
aptr
2 15 27 38
bptr
1 2 13 15 24
cptr
1 13 24 26
aptr
2 15 27 38
bptr
1 2 13 15 24 26
cptr
1 13 24 26
aptr
2 15 27 38
bptr
1 2 13 15 24 26 27 38
cptr

Mergesort

- The mergesort algorithm is therefore easy to describe. If n = 1, there is only one element to sort, and the answer is at hand. Otherwise, recursively mergesort the first half and the second half. This gives two sorted halves, which can then be merged together using the merging algorithm described above.

- This algorithm is a classic divide-and-conquer strategy.

Mergesort Algorithm

To sort an array of A[p . . r]:

Divide

Divide the n-element sequence to be sorted into two subsequences of n/2 elements each

Conquer

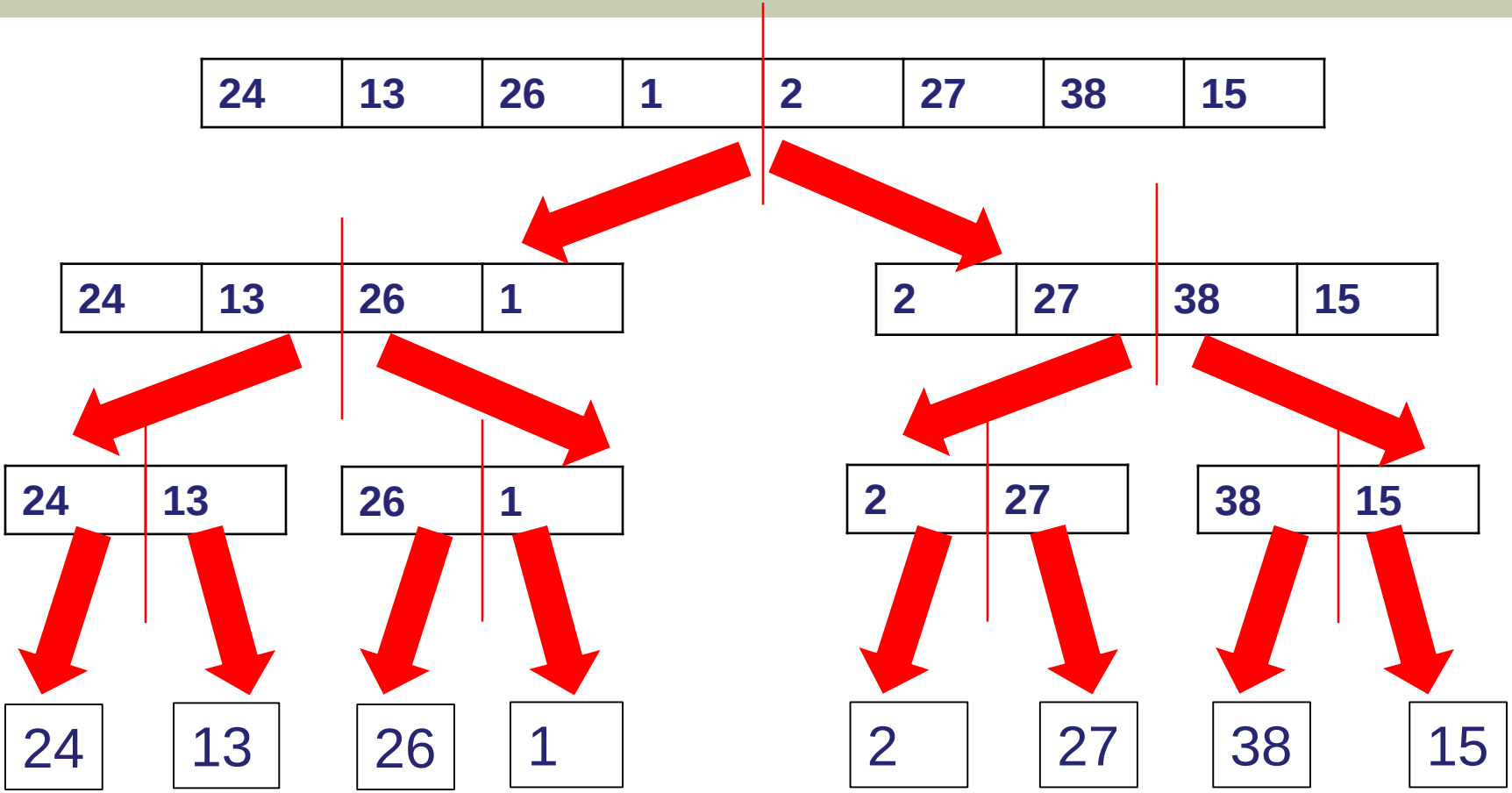
Sort the subsequences recursively using merge sort

When the size of the sequences is 1 there is nothing more to do

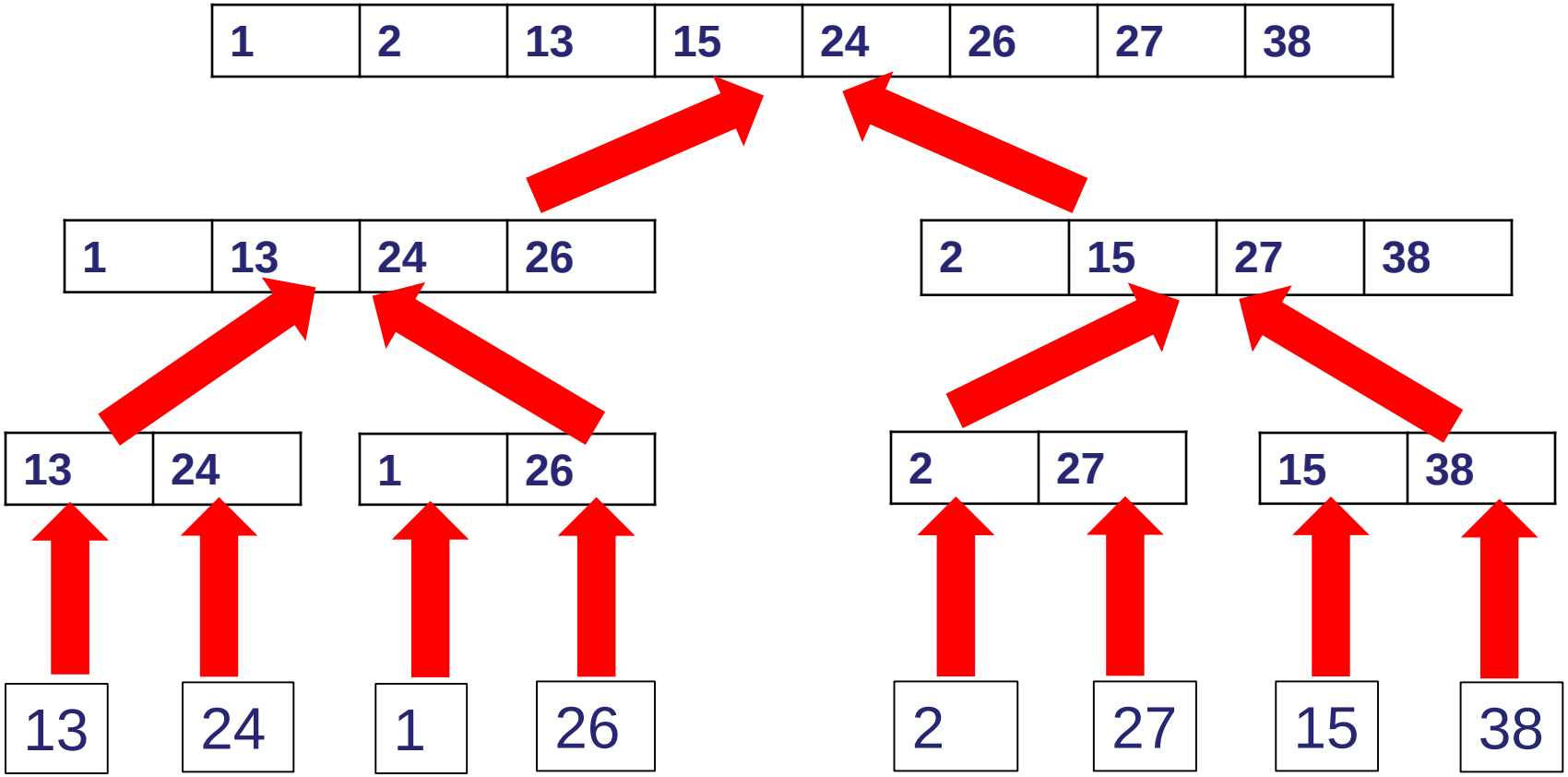
Combine

Merge the two sorted subsequences

Mergesort


24 13 26 1 2 27 38 15
24 13 26 1
2 27 38 15
24 13
26 1
2 27
38 15
24
13
26
1
2
27
38
15

Mergesort


1 2 13 15 24 26 27 38
1 13 24 26
2 15 27 38
13 24
1 26
2 27
15 38
13
24
1
26
2
27
15
38

C Code of Mergesort

```
void MSort(ElementType A[ ], ElementType TmpArray[ ], int Left, int Right )
{       int Center;

        if( Left < Right )
        {   Center = ( Left + Right ) / 2;
            MSort( A, TmpArray, Left, Center );
            MSort( A, TmpArray, Center + 1, Right );
            Merge( A, TmpArray, Left, Center + 1, Right );
        }
 }

void Mergesort( int A[ ], int N )
{         int *TmpArray;

        TmpArray = malloc( N * sizeof( ElementType ) );
        if( TmpArray != NULL )
        {   MSort( A, TmpArray, 0, N - 1 );
            free( TmpArray );
        }
        else
        FatalError( "No space for tmp array!!!" );
}
```


Figure 7.9 Mergesort routine

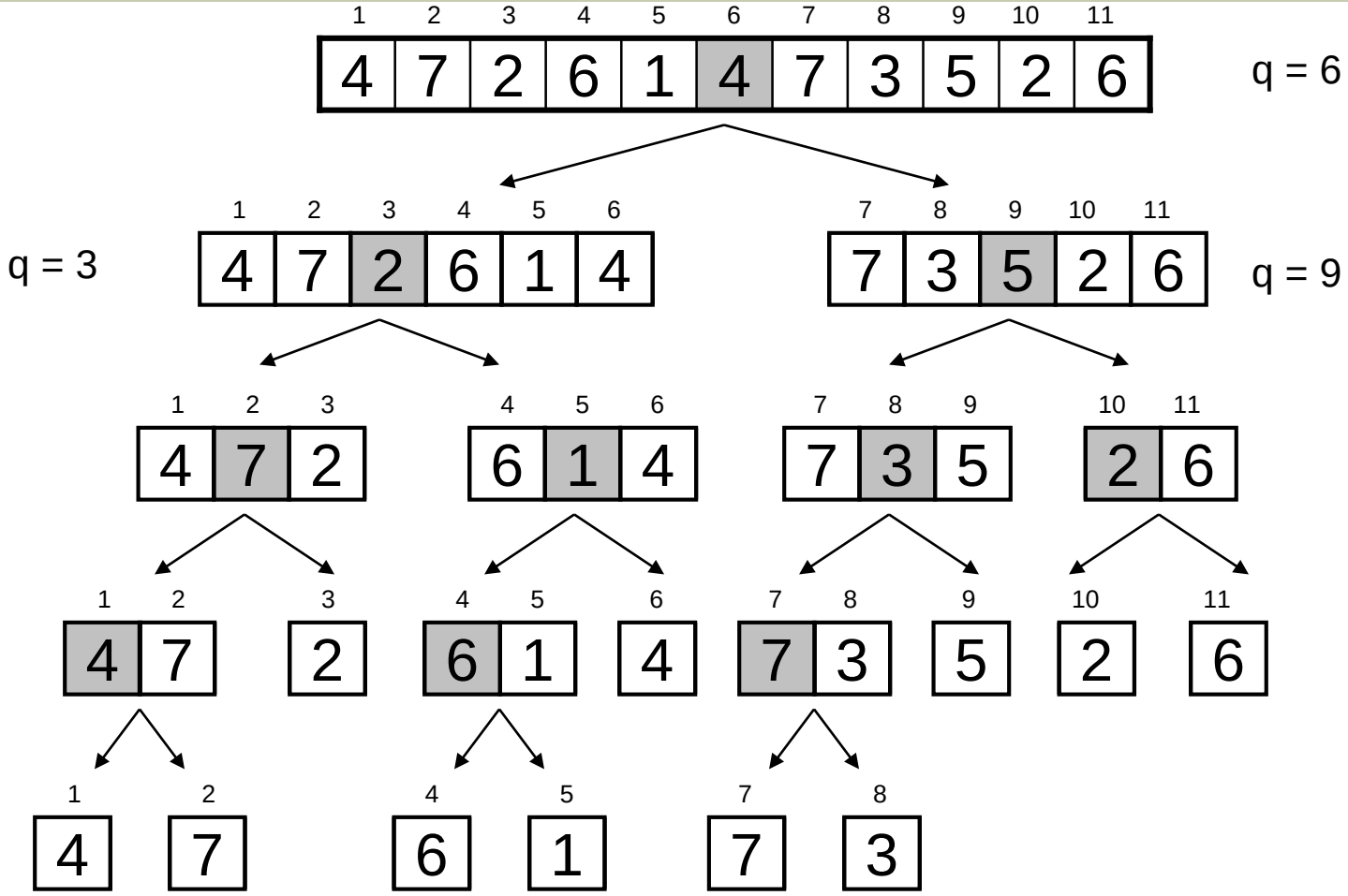
C Code of Mergesort

```
void  Merge( ElementType A[ ], ElementType TmpArray[ ], int Lpos, int Rpos,
                                   int RightEnd )
{       int i, LeftEnd, NumElements, TmpPos;

        LeftEnd = Rpos - 1;
        TmpPos = Lpos;
        NumElements = RightEnd - Lpos + 1;

              while( Lpos <= LeftEnd && Rpos <= RightEnd )
               if( A[ Lpos ] <= A[ Rpos ] )
                                   TmpArray[ TmpPos++ ] = A[ Lpos++ ];
                        else
                                   TmpArray[ TmpPos++ ] = A[ Rpos++ ];

              while( Lpos <= LeftEnd )  /* Copy rest of first half */
              TmpArray[ TmpPos++ ] = A[ Lpos++ ];
              while( Rpos <= RightEnd ) /* Copy rest of second half */
               TmpArray[ TmpPos++ ] = A[ Rpos++ ];

              for( i = 0; i < NumElements; i++, RightEnd-- )
                        A[ RightEnd ] = TmpArray[ RightEnd ];
}
```


Figure 7.10 Mergesort routine

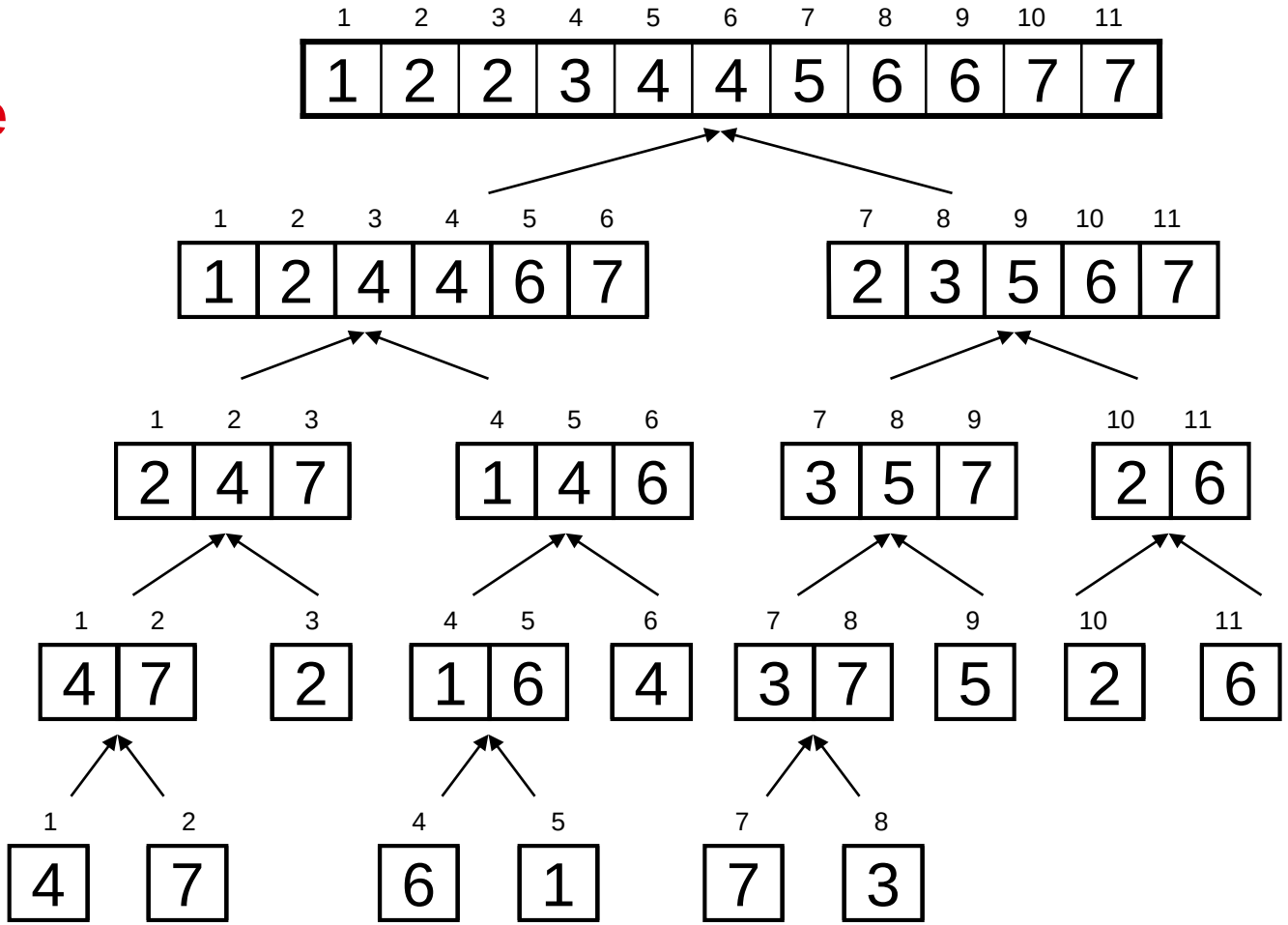
Example – n not a Power of 2

Divide

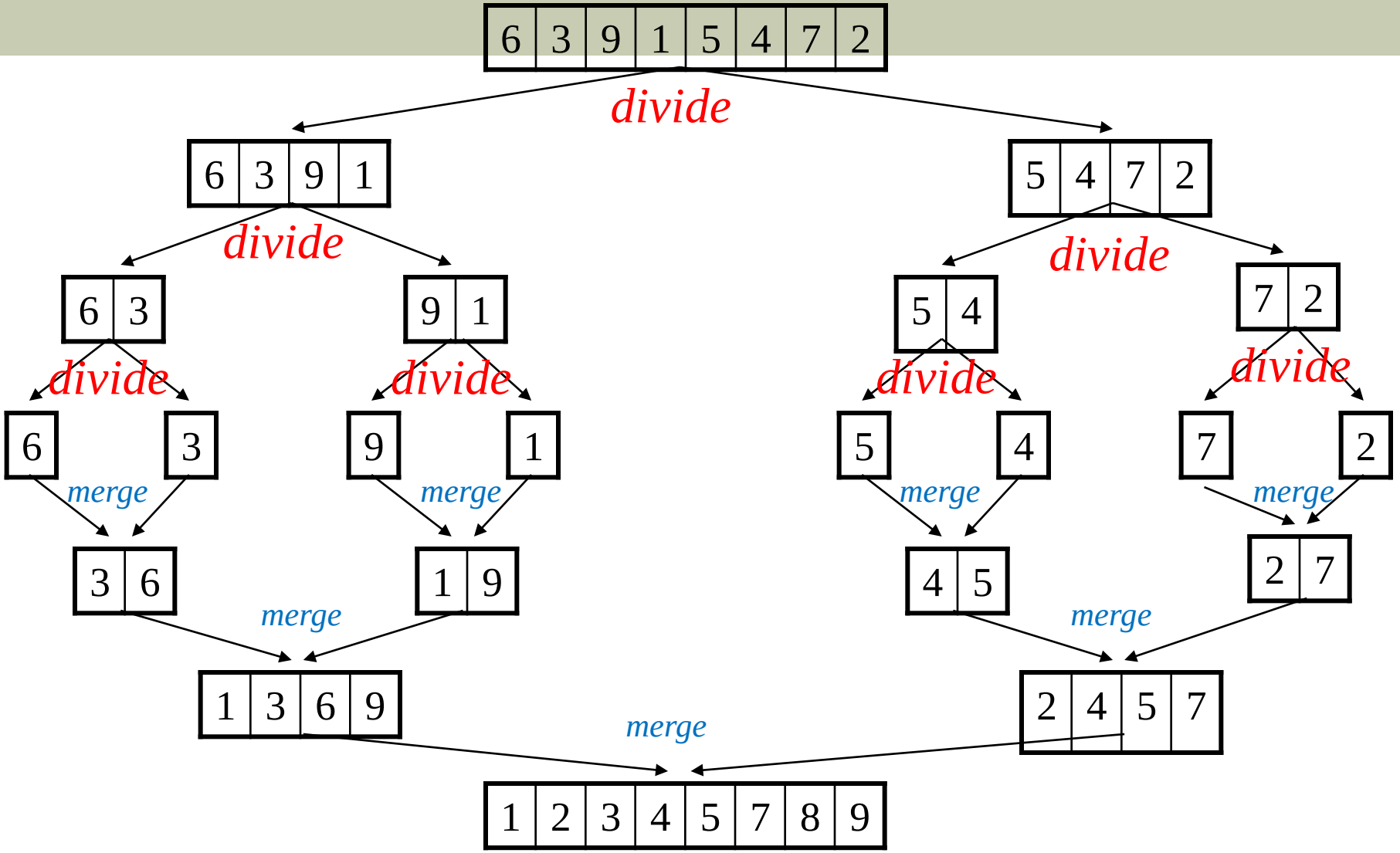


Example – n Not a Power of 2

Conquer and Merge

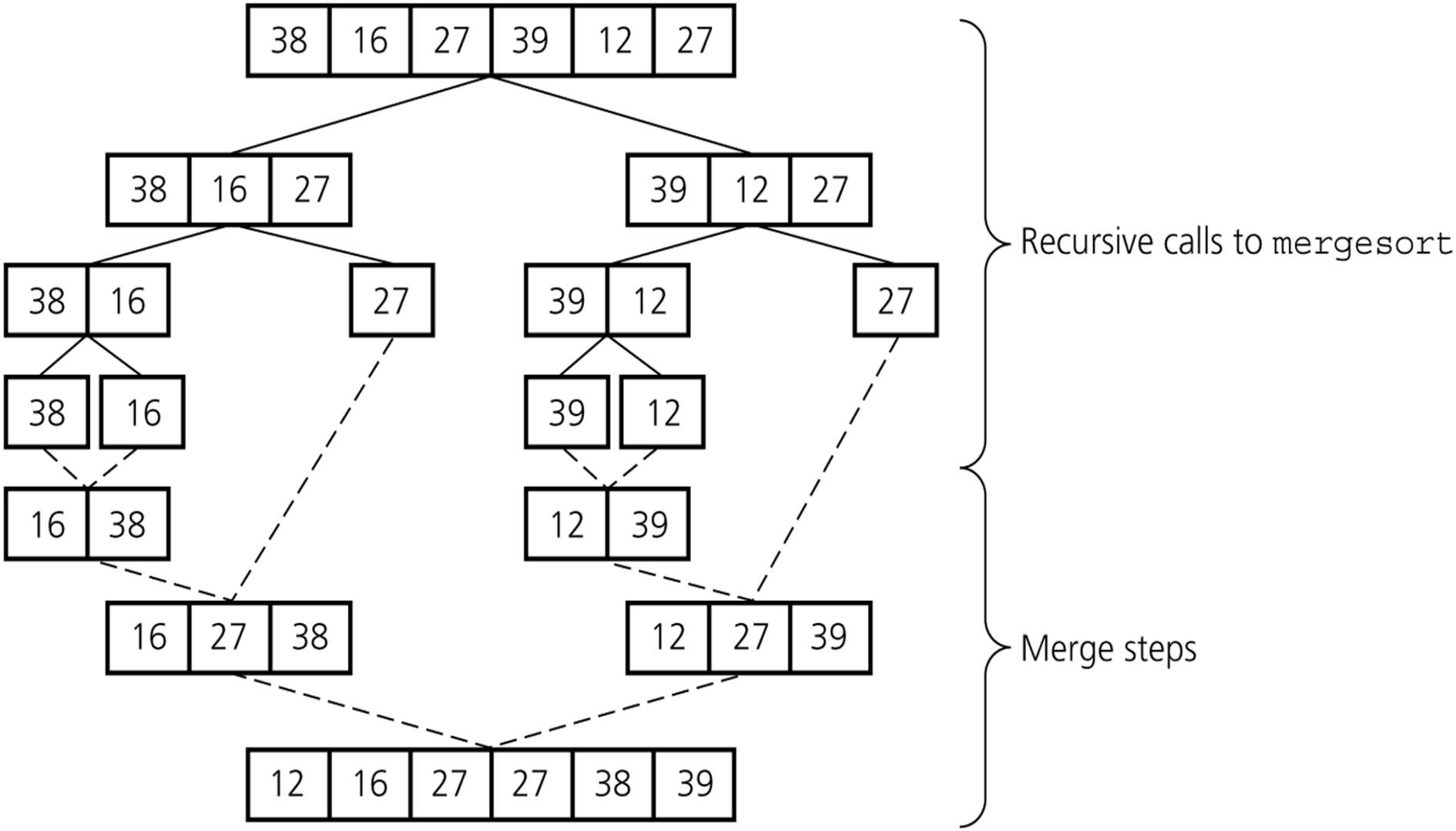


Mergesort - Example



Designed by Ayla SAYLI and Armagan ELIBOL

Mergesort – Example



Analysis of Mergesort

We will assume that n is a power of 2, so that we always split into even halves. For n = 1, the time to mergesort is constant, which we will denote by 1. Otherwise, the time to mergesort n numbers is equal to the time to do two recursive mergesorts of size n/2, plus the time to merge, which is linear.
The equations below say this exactly:

$$T(1) = 1$$

$$T(n) = 2T(n/2) + n$$

Analysis of Mergesort

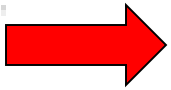
This is a standard recurrence relation, which can be solved several ways. We will show two methods.

- The first idea is to divide the recurrence relation through by n. This yields

$$\frac{T(n)}{n} = \frac{T(n/2)}{n/2} + 1$$

This equation is valid for any n that is a power of 2, so we may also write

$$\frac{T(n/2)}{n/2} = \frac{T(n/4)}{n/4} + 1$$



$$\frac{T(n/4)}{n/4} = \frac{T(n/8)}{n/8} + 1$$

$$\cdots$$

$$\frac{T(2)}{2} = \frac{T(1)}{1} + 1$$

Analysis of Mergesort

Now add up all the equations:

$$\frac{T(n)}{n} = \frac{T(1)}{1} + \log n$$

The final answer is

$$T(n) = n \log n + n = O(n \log n)$$

Analysis of Mergesort

- An alternative method is to substitute the recurrence relation continually on the right-hand side.

$$T(n) = 2T(n/2) + n$$

Since we can substitute $n/2$ into the main equation,

$$2T(n/2) = 2(2(T(n/4)) + n/2) = 4T(n/4) + n$$

we have

$$T(n) = 4T(n/4) + 2n$$

Again, by substituting $n/4$ into the main equation, we see that

$$4T(n/4) = 4(2T(n/8)) + (n/4) = 8T(n/8) + n$$

So we have $T(n) = 8T(n/8) + 3n$

Continuing in this manner, we obtain $T(n) = 2^k T(n/2^k) + kn$

Using $k = \log n$, we obtain

$$T(n) = nT(1) + n \log n = n \log n + n = O(n \log n)$$

Quicksort

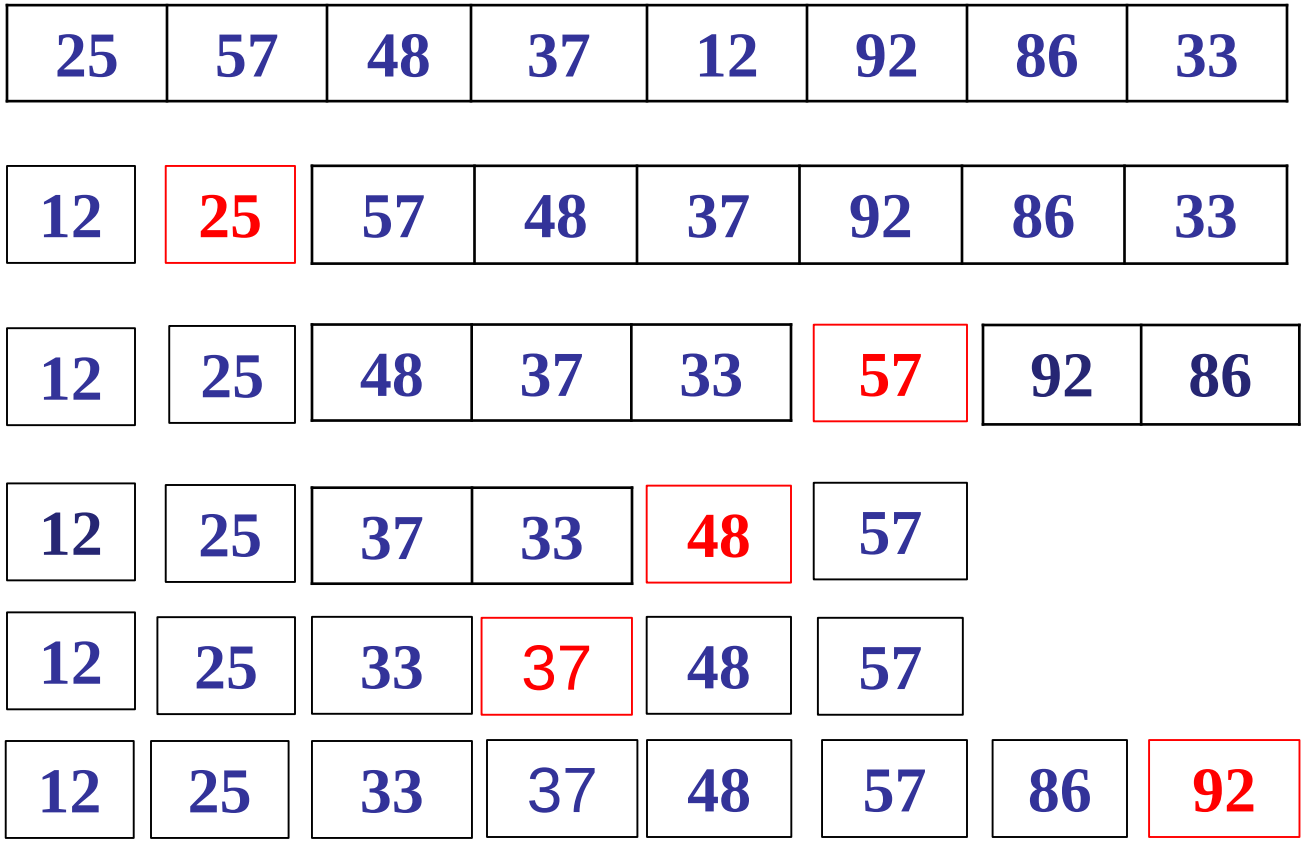
The basic algorithm to sort an array S consists of the following four steps:

- If the number of elements in S is 0 or 1, then return.
- Pick any element v in S. This is called the pivot.
- Partition S – {v} (the remaining elements in S) into two disjoint groups:

$$S_1 = \{x \in S - \{v\} \mid x < v\} \text{ and } S_2 = \{x \in S - \{v\} \mid x > v\}$$

- Return {quicksort(S_1) followed by v followed by quicksort(S_2)}

Quicksort


25 57 48 37 12 92 86 33
12 25 57 48 37 92 86 33
12 25 48 37 33 57 92 86
12 25 37 33 48 57
12 25 33 37 48 57
12 25 33 37 48 57 86 92

Quicksort – Picking the Pivot

Although the algorithm works no matter which element is chosen as pivot, some choices can be better than others.

- If the input is random, the first element can be a pivot, otherwise it is a wrong way to select the first element but the popular one....
- A safe course is merely to choose the pivot randomly. But random number generation is generally expensive...
- Median-of-Three Partitioning:

$$\lfloor (the\ first\ element\ +\ the\ last\ element)\ /\ 2 \rfloor$$

C Code of Quicksort

```
ElementType Median3( ElementType A[ ], int Left, int Right )
    {
      int Center = ( Left + Right ) / 2;

      if( A[ Left ] > A[ Center ] )
        Swap( &A[ Left ], &A[ Center ] );
      if( A[ Left ] > A[ Right ] )
        Swap( &A[ Left ], &A[ Right ] );
      if( A[ Center ] > A[ Right ] )
        Swap( &A[ Center ], &A[ Right ] );

      /* Invariant: A[ Left ] <= A[ Center ] <= A[ Right ] */

      Swap( &A[ Center ], &A[ Right - 1 ] );  /* Hide pivot */
      return A[ Right - 1 ];               /* Return pivot */
    }
```

$T(N) = O(N \log N)$

See Figure 7.14 and Figure 7.16 (page 241-246)

Radix Sort

Example: The input is 64, 8, 216, 512, 27, 729, 0, 1, 343, 125

0	1	512	343	64	125	216	27	8	729
0	1	2	3	4	5	6	7	8	9

8		729							
1	216	27							
0	512	125		343		64			
0	1	2	3	4	5	6	7	8	9

64									
27									
8									
1									
0	125	216	343		512		729		
0	1	2	3	4	5	6	7	8	9

T(N) = O(N)

Designed by Ayla SAYLI and Armagan ELIBOL

Comparison of Sorting Algorithms

	Worst case	Average case
Selection sort	n^2	n^2
Bubble sort	n^2	n^2
Insertion sort	n^2	n^2
Mergesort	n * log n	n * log n
Quicksort	n^2	n * log n
Radix sort	n	n
Treesort	n^2	n * log n
Heapsort	n * log n	n * log n

Questions and Answers

Thank you ...

Prof. Dr. Ayla ŞAYLI

http://avesis.yildiz.edu.tr/sayli/
sayli@yildiz.edu.tr